WORKFLOW_STATE_MACHINE

Status: Draft v1. Matches PRODUCT_SPEC flow F1 to F14 (no baseline, no code execution).

1. States

State	Meaning	Who moves it forward
CREATED	Repo chosen and verified, SHA pinned	API
SNAPSHOTTING	Worker cloning at SHA (temporary copy) and creating the work branch on GitHub	Worker
SNAPSHOTTED	Temporary snapshot ready and work branch created on GitHub	Worker
ANALYZING	Content-based detection running	Worker
ANALYZED	Project profile saved	Worker
FINDING	Rules and AI producing findings	Worker
FINDINGS_READY	User can read findings	Worker
PLANNING	AI and rules producing plan suggestions	Worker
PLAN_READY	User can select operations	Worker
APPROVED	Approval record saved for selected operations	API (user action)
EXECUTING	Worker applying approved edits and committing them to the work branch	Worker
V2_READY	Version 2 committed to the work branch and manifest created	Worker
CHECKING	Static checks running	Worker
IN_REVIEW	User sees diff, manifest, check results	Worker
PUSH_PERMITTED	User gave explicit permission to open the pull request	API (user action)

State	Meaning	Who moves it forward
PUSHING	Backend verifying branch and SHA, then opening the PR	Backend
PUSHED	PR opened from the work branch. Terminal.	Backend
FAILED	Step failed, error code saved. Terminal.	Any
CANCELLED	User cancelled. Terminal.	API
STALE	Repo SHA changed after pinning. Terminal for this workflow.	Any

Terminal states: PUSHED, FAILED, CANCELLED, STALE. Note: PUSH_PERMITTED, PUSHING and PUSHED now mean permission to open the PR, opening the PR, and PR opened, because commits go to the work branch during EXECUTING.

2. Allowed transitions

CREATED -> SNAPSHOTTING -> SNAPSHOTTED -> ANALYZING -> ANALYZED -> FINDING -> FINDINGS_READY -> PLANNING -> PLAN_READY -> APPROVED -> EXECUTING -> V2_READY -> CHECKING -> IN_REVIEW -> PUSH_PERMITTED -> PUSHING -> PUSHED

Extra transitions:

- PLAN_READY -> PLAN_READY (user changes selection, no approval yet)
- APPROVED -> PLAN_READY (user withdraws approval before execution starts)
- IN_REVIEW -> PLAN_READY (user wants to change plan; creates new approval later, old approval marked superseded)
- Any non-terminal state -> FAILED, CANCELLED or STALE
- No transition out of a terminal state. Starting again means a new workflow from zero.

Any transition not in this list is rejected.

3. Guards (checked in code, never by AI)

Transition	Guard
CREATED	Owner equals logged-in user, repo public, personal repo, user active, daily limit not exceeded
PLAN_READY -> APPROVED	Same user, current state PLAN_READY, plan sourceSha equals workflow sourceSha, selected operations valid, extra confirmation present for high-risk or delete
APPROVED -> EXECUTING	Approval status valid, sourceSha unchanged
EXECUTING -> V2_READY	Every changed file maps to an approved operation
CHECKING -> IN_REVIEW	diff equals manifest, no unapproved change
IN_REVIEW -> PUSH_PERMITTED	Explicit user action, same user, approval still valid
PUSH_PERMITTED -> PUSHING	Remote SHA still equals sourceSha, repo still public and owned by user, work branch is not main and its head equals Obsidian's last commit
PUSHING -> PUSHED	PR created from the work branch

4. How a transition is done

One MongoDB updateOne with filter (_id, userId, state equals expected, version equals expected) and update (new state, version plus 1, updatedAt). If it matches 0 documents, the request is rejected as a conflict. This stops two clicks or two workers from both moving the same workflow.

5. Idempotency

- Approve: unique key (workflowId, planHash). Repeat click returns the existing approval.
- Execute job id: workflowId plus approvalId. A duplicate job is ignored.
- PR: if pr is already set, return it and never open a second PR. Commits to the work branch are tracked by commit SHA so a retry does not duplicate them.

- Each worker step checks it is in the expected -ING state before work and writes its result once.

6. Failure, retry and recovery

Situation	Behaviour
Worker crash mid-step	Job retried (limited attempts, backoff). Step is written to be repeatable: it cleans its own partial output first.
Retries exhausted	FAILED with error code. Temp files removed.
Timeout	Step fails with STEP_TIMEOUT.
Redis lost	On startup, scan workflows in -ING states and re-enqueue.
Stuck -ING state	Watchdog marks FAILED after max time.
AI failure	FINDING and PLANNING continue with rule-based results only and a warning. Not a failure.
SHA changed	STALE. Approvals marked stale.
User cancels	CANCELLED. Running job gets a cancel flag checked between steps. Cleanup follows.
User deletes data during active job	Cancel first, wait for worker to stop, then delete files and records, then verify deletion.

7. Stale approval rules

An approval is valid only for: its user, its workflow, its sourceSha and its approvedOpIds with planHash. If any of these differ, the approval is stale and rejected. A past approval never allows unrelated changes.

8. Concurrency rules

Only one non-terminal workflow per user and repo (partial unique index). Per-user cap on simultaneous workflows. One worker step per workflow at a time.

9. Tests for this document

- Every disallowed transition is rejected (table-driven test).
- Two simultaneous approve calls produce one approval.
- Two simultaneous PR calls produce one PR.
- Crash during EXECUTING then retry gives the same V2.
- SHA change before opening the PR gives STALE and no PR.
- Cancel during any -ING state ends in CANCELLED and cleans files.